UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 2
SEGUNDO SEMESTRE 2026
PROYECTO 2

Andy Jefferson González Fuentes 202231338

MANUAL DE USUARIO

1. Introducción

Este manual describe el uso del IDE desarrollado para el Proyecto 1 del curso de Compiladores 2. La aplicación permite crear, editar y analizar programas escritos en tres lenguajes definidos para el curso: PigLatin, Y y Zetariano (Z). El IDE incluye un editor de código con resaltado de sintaxis, un árbol de trabajo para gestionar proyectos y archivos, y un analizador que detecta errores léxicos, sintácticos y semánticos, mostrándolos tanto en una tabla como directamente sobre las líneas del código.

El objetivo de este documento es guiar al usuario paso a paso en la instalación, ejecución y uso de cada una de las funciones disponibles en la interfaz.

2. Requisitos del sistema

Para ejecutar el IDE en su equipo, necesita lo siguiente:

- Java Development Kit (JDK) 17 o superior.
- Apache Maven (para compilar y ejecutar el proyecto).
- Conexión a internet la primera vez que se compile, para descargar las dependencias (JavaFX 21 y ANTLR 4.13).
- Sistema operativo: Windows, Linux o macOS con soporte para JavaFX.
- Al menos 4 GB de RAM disponibles y 500 MB de espacio en disco.

3. Instalación y ejecución

Siga estos pasos para poner en marcha el IDE:

- Clonar o descargar el repositorio del proyecto:

```
git clone https://github.com/AndyGonzalez202231338/COMPI2_PROYECTO1_SS2026.git
```

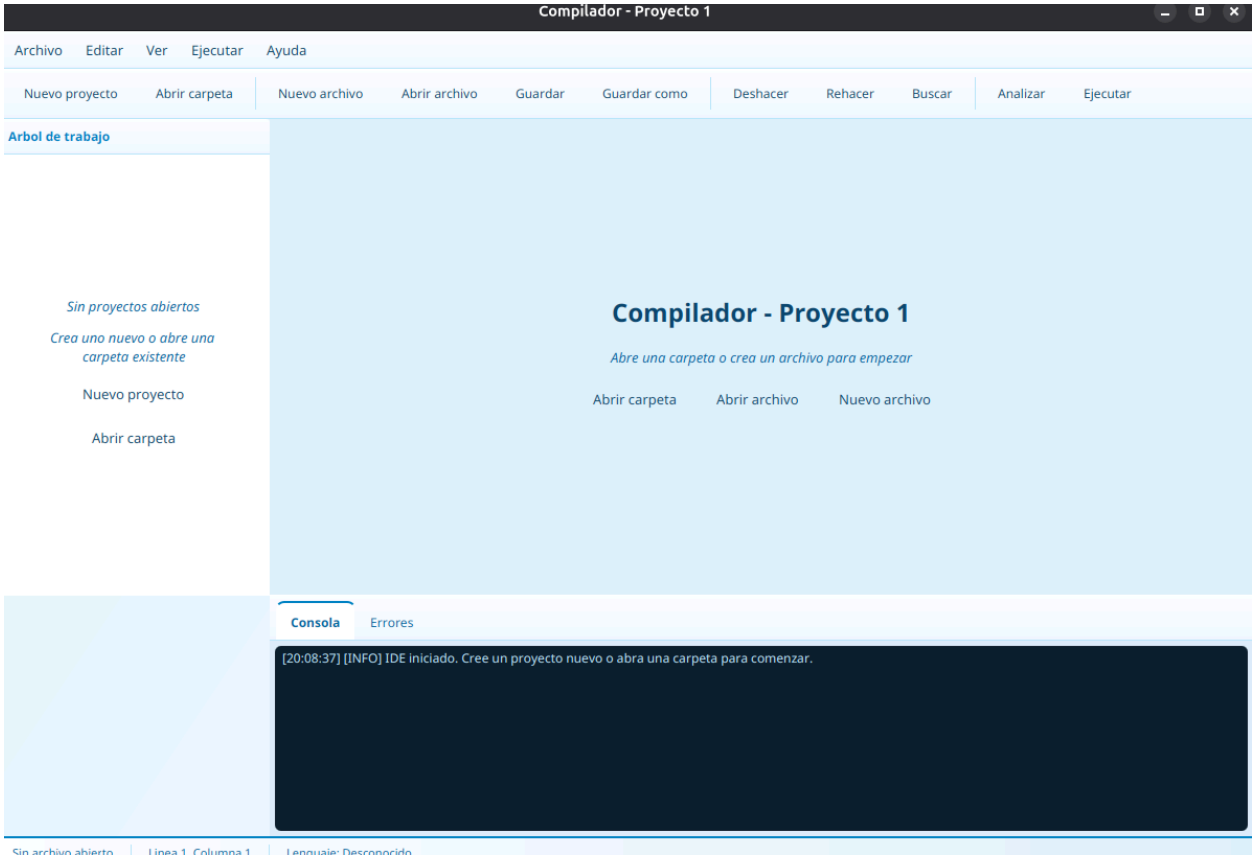
- Ingresar a la carpeta del proyecto (Proyecto1), donde se encuentra el archivo pom.xml.
- Ejecutar el siguiente comando en una terminal para compilar y abrir la aplicación:

```
mvn javafx:run
```

- También puede abrir la carpeta Proyecto1 como proyecto Maven desde un IDE (NetBeans, IntelliJ o Eclipse) y ejecutar la clase MainApp.

4. Descripción general de la interfaz

Al iniciar la aplicación se muestra la ventana principal del IDE, dividida en las siguientes áreas: barra de menús, barra de herramientas, árbol de trabajo (panel izquierdo), área de edición con pestañas (panel central), panel inferior de consola/errores y barra de estado.



4.1 Barra de menús

La barra de menús ofrece las siguientes opciones:

Menú	**Contiene**
Archivo	Nuevo archivo, Nuevo proyecto, Abrir archivo, Abrir carpeta, Guardar, Guardar como, Cerrar pestaña, Salir
Editar	Deshacer, Rehacer, Cortar, Copiar, Pegar, Seleccionar todo, Buscar / Reemplazar
Ver	Mostrar/ocultar árbol de trabajo, Mostrar/ocultar consola y errores

Menú	Contiene
Ejecutar	Analizar archivo actual, Compilar todo el proyecto, Ejecutar último análisis
Ayuda	Acerca de, Manual de usuario

4.2 Barra de herramientas

Debajo del menú se encuentra la barra de herramientas con accesos directos a las acciones más utilizadas: Nuevo proyecto, Abrir carpeta, Nuevo archivo, Abrir archivo, Guardar, Guardar como, Deshacer, Rehacer, Buscar, Analizar y Ejecutar.

4.3 Árbol de trabajo

Muestra las carpetas de proyecto abiertas y sus archivos. Se pueden tener varios proyectos abiertos al mismo tiempo. Al hacer clic derecho sobre el fondo del árbol aparece un menú para crear un nuevo proyecto o abrir una carpeta. Al hacer clic derecho sobre un archivo o carpeta específica aparece un menú contextual con las acciones disponibles para ese elemento.

Si no hay ningún proyecto abierto, el panel muestra un mensaje de bienvenida con botones para crear un proyecto nuevo o abrir una carpeta existente.

4.4 Área de edición

Cada archivo abierto se muestra en una pestaña independiente dentro del panel central. Se pueden tener varios archivos abiertos simultáneamente y cambiar entre ellos haciendo clic en su pestaña.

4.5 Panel inferior: Consola y Errores

Este panel tiene dos pestañas: "Consola", donde se registran mensajes informativos, advertencias y errores con marca de hora; y "Errores", una tabla con el detalle de cada error encontrado al analizar un archivo. El panel puede mostrarse u ocultarse desde el menú Ver.

4.6 Barra de estado

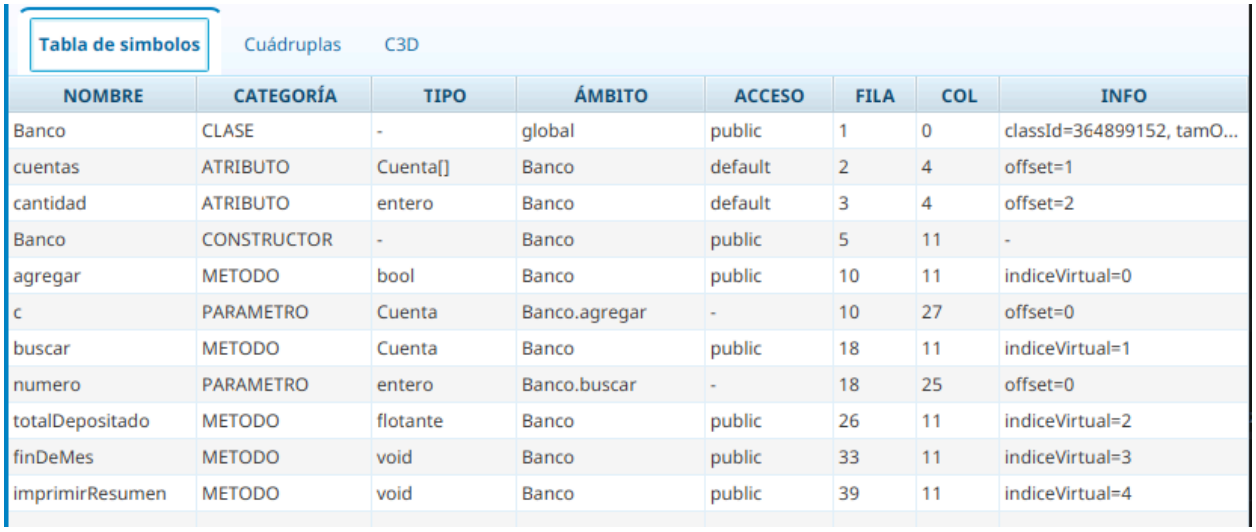
En la parte inferior de la ventana se muestra la ruta del archivo activo, la posición del cursor (línea y columna), el lenguaje detectado según la extensión del archivo y si el archivo tiene cambios sin guardar.

4.7 Botón "RISC-V"

En la barra de herramientas se agrega un nuevo botón "RISC-V" (o en el menú Ejecutar > Generar RISC-V). Al hacer clic, el sistema analiza el archivo activo (o el proyecto completo si es un .pig con imports), genera el código intermedio C3D, traduce las cuádruplas a ensamblador RISC-V y produce un archivo .s listo para ejecutarse en RARS. El archivo generado se guarda en la carpeta de salida del proyecto.

4.8 Vista de tabla de símbolos

Al completar el análisis semántico, se habilita una vista de tabla de símbolos accesible desde el panel inferior o desde el menú Ver. La tabla muestra la información de cada símbolo detectado durante el análisis: nombre, categoría (variable, función, clase, atributo, parámetro, estructura, campo, constructor, método), tipo de dato, ámbito al que pertenece, línea y columna donde se declaró, y si fue inicializado. Para clases y estructuras se muestra además el tamaño del objeto y los offsets de cada campo.



NOMBRE	CATEGORÍA	TIPO	ÁMBITO	ACCESO	FILA	COL	INFO
Banco	CLASE	-	global	public	1	0	classId=364899152, tamO...
cuentas	ATRIBUTO	Cuenta[]	Banco	default	2	4	offset=1
cantidad	ATRIBUTO	entero	Banco	default	3	4	offset=2
Banco	CONSTRUCTOR	-	Banco	public	5	11	-
agregar	METODO	bool	Banco	public	10	11	indiceVirtual=0
c	PARAMETRO	Cuenta	Banco.agregar	-	10	27	offset=0
buscar	METODO	Cuenta	Banco	public	18	11	indiceVirtual=1
numero	PARAMETRO	entero	Banco.buscar	-	18	25	offset=0
totalDepositado	METODO	flotante	Banco	public	26	11	indiceVirtual=2
finDeMes	METODO	void	Banco	public	33	11	indiceVirtual=3
imprimirResumen	METODO	void	Banco	public	39	11	indiceVirtual=4

4.9 Vista de cuádruplas (C3D)

Después de la generación de código intermedio, se habilita una tabla que muestra las cuádruplas generadas. Cada fila representa una instrucción de tres direcciones con las columnas: número de cuádrupla, operación, operando 1, operando 2 y resultado. Las operaciones incluyen asignaciones, operaciones aritméticas, saltos condicionales e incondicionales, llamadas a función, accesos a campos y arreglos, entre otras. Esta vista permite inspeccionar paso a paso cómo el compilador tradujo el código fuente a instrucciones intermedias.



#	OPERADOR	OPERANDO 1	OPERANDO 2	RESULTADO
0	begin_func	main	0	-
1	println	"Hola Mundo"	-	-
2	end_func	-	-	-

4.10 Vista del código C3D (texto)

Junto a la tabla de cuádruplas, se ofrece una vista de texto plano con la representación legible del código intermedio generado. Cada línea muestra una cuádrupla en formato legible (por ejemplo: "t0 = a + b", "if_false t1 goto L3", "param t2", "call funcion, 2 -> t3"). Esta vista facilita la depuración y permite copiar el C3D para análisis externo.

Tabla de simbolos | Cuádruplas | C3D

```
0: begin_func main, 0
1: println "Hola Mundo"  [cadena]
2: end_func
```

4.11 Vista del código RISC-V (.s)

Al generar el código RISC-V, se muestra el archivo .s completo en un panel de texto con resaltado. El código incluye la sección .data (literales de cadena y flotantes), la zona del heap (heap_ptr y heap_mem) y la sección .text con el código de cada función (prólogo, cuerpo traducido y epílogo). El usuario puede copiar este código o guardarlo para ejecutarlo directamente en RARS.

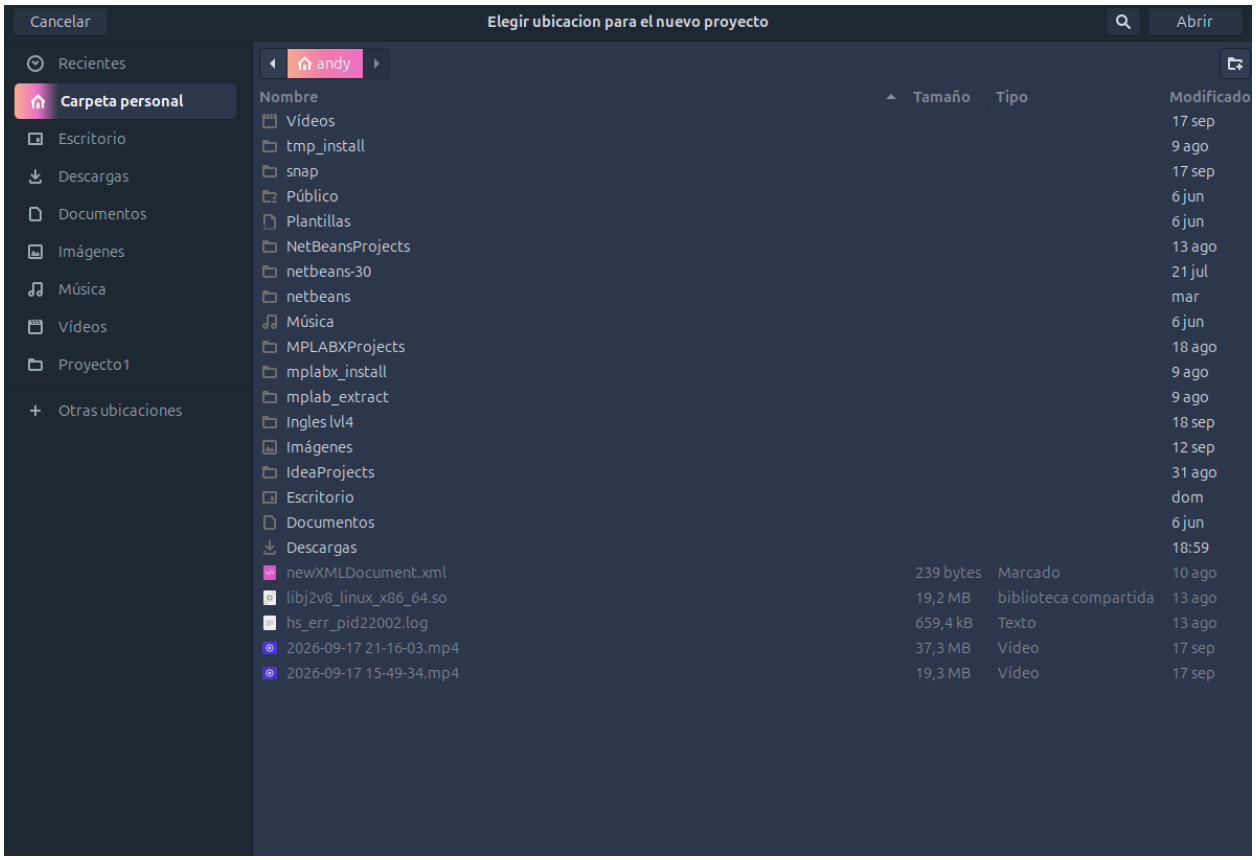
Minimo.s ×

```
1   # Codigo RISC-V (RV32I + F) generado automaticamente
2   # Pensado para el emulador RARS
3
4   .data
5   str_0: .string "Hola Mundo"
6
7   # --- heap: memoria para new (objetos, estructuras y arreglos) ---
8   heap_ptr: .word 0
9   heap_mem: .space 65536
10
11  .text
12  .globl main
13
14  main:
15      # inicio del heap: heap_ptr = direccion de heap_mem
16      la t0, heap_mem
17      la t1, heap_ptr
18      sw t0, 0(t1)
19      # --- prologo de main (frame = 16 bytes) ---
20      addi sp, sp, -16
21      sw ra, 12(sp)
22      sw fp, 8(sp)
23      addi fp, sp, 16
```

5. Gestión de proyectos y archivos

5.1 Crear un nuevo proyecto

Vaya a Archivo > Nuevo proyecto (o use el botón "Nuevo proyecto" de la barra de herramientas). Seleccione la carpeta donde se creará el proyecto y escriba el nombre de la nueva carpeta. El proyecto se agregará automáticamente al árbol de trabajo.



5.2 Abrir una carpeta como proyecto

Vaya a Archivo > Abrir carpeta y seleccione la carpeta que desea agregar como proyecto al árbol de trabajo. Puede repetir esta acción para tener varios proyectos abiertos a la vez.

5.3 Crear un nuevo archivo

Vaya a Archivo > Nuevo archivo. Elija la ubicación y el nombre del archivo, indicando la extensión correspondiente al lenguaje que desea usar (.pig, .y o .z). El archivo se crea vacío y se abre automáticamente en una nueva pestaña.

5.4 Abrir un archivo existente

Vaya a Archivo > Abrir archivo y seleccione el archivo deseado, o bien haga doble clic sobre él directamente en el árbol de trabajo.

5.5 Guardar cambios

Use Archivo > Guardar (o el botón correspondiente) para guardar el archivo activo. Use Archivo > Guardar como para guardar el contenido actual en una nueva ruta o con otro nombre.

5.6 Cerrar archivos

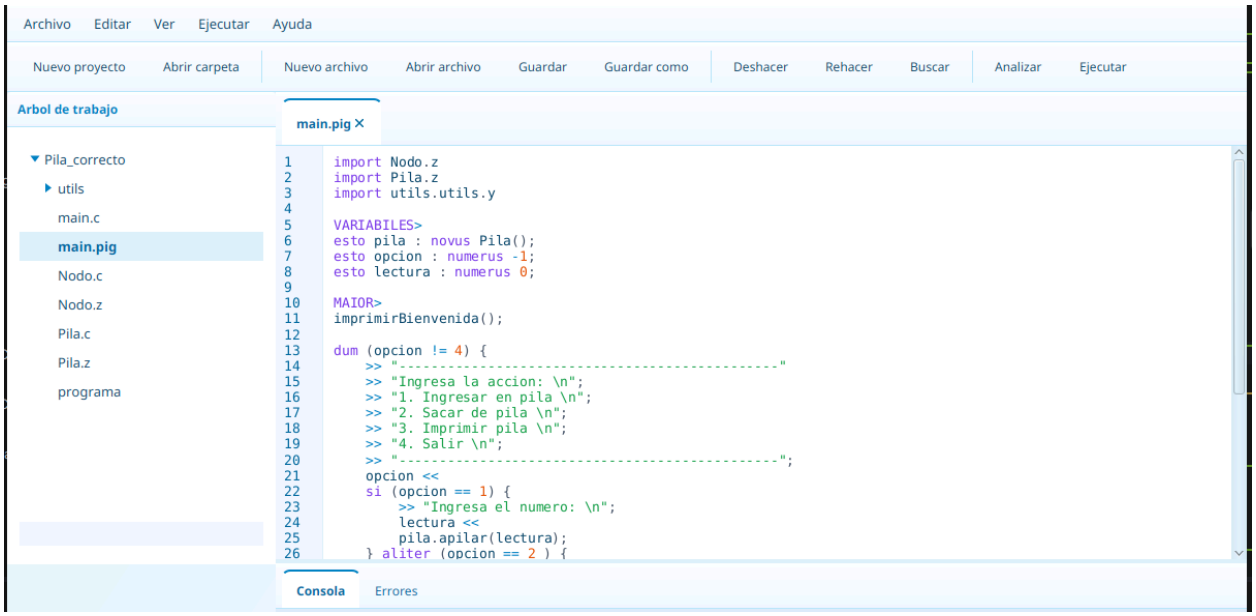
Cada pestaña tiene un botón para cerrarla. Si el archivo tiene cambios sin guardar, el sistema solicitará confirmación antes de cerrarlo.



6. El editor de código

El editor incorporado ofrece resaltado de sintaxis según el lenguaje detectado por la extensión del archivo, además de las siguientes funciones disponibles desde el menú Editar o la barra de herramientas:

- Deshacer / Rehacer cambios.
- Cortar, copiar y pegar texto.
- Seleccionar todo el contenido del archivo.
- Buscar y reemplazar texto dentro del archivo activo.



7. Lenguajes soportados

El IDE reconoce automáticamente el lenguaje de un archivo según su extensión. Actualmente se soportan tres lenguajes definidos para el curso:

Extensión	Lenguaje	Descripción general
.pig	PigLatin	Lenguaje base del curso; sus sentencias terminan en punto y coma y utiliza ">>" para imprimir valores.
.y	Y	Lenguaje estructurado por indentación (similar a Python), organizado en secciones %estructuras y %funciones.
.z	Zetariano (Z)	Lenguaje orientado a objetos con clases, atributos, constructores y métodos, y las instrucciones print/println/readln.

La sintaxis detallada de cada lenguaje corresponde a la definición entregada en el curso; se recomienda conservar a la mano el material de la gramática de cada lenguaje al momento de escribir el código.

8. Análisis de código

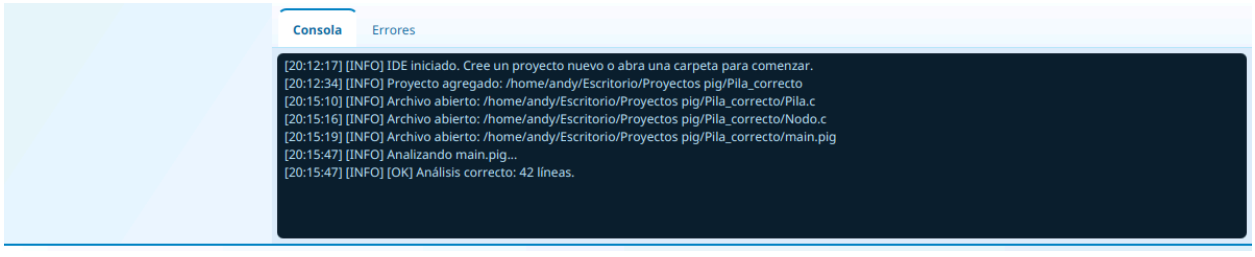
El botón "Analizar" (o el menú Ejecutar > Analizar archivo actual) ejecuta el análisis léxico, sintáctico y semántico del archivo abierto en la pestaña activa.

8.1 Cómo analizar un archivo

- Abra o cree el archivo que desea analizar.
- Haga clic en el botón "Analizar" para ejecutar el análisis léxico, sintáctico y semántico.
- Si el análisis no reporta errores, se genera automáticamente el código intermedio (C3D) y se habilitan las vistas de cuádruplas y tabla de símbolos.
- Haga clic en el botón "RISC-V" para generar el archivo ensamblador (.s) a partir del C3D.
- Revise los resultados en las pestañas "Consola", "Errores", "Cuádruplas", "Tabla de símbolos" y "RISC-V".

8.2 Consola de salida

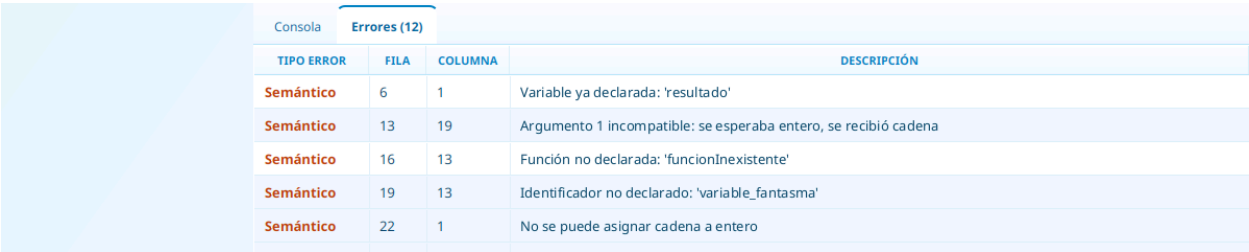
La consola muestra, con marca de hora, mensajes de tipo INFO, ADVERTENCIA y ERROR generados durante el análisis y demás acciones del IDE (por ejemplo, al crear archivos o proyectos).



8.3 Tabla de errores

Cada error detectado se agrega como una fila con las columnas: Tipo de error, Fila, Columna y Descripción. Los tipos de error posibles son:

- Léxico: caracteres o símbolos no reconocidos por el lenguaje.
- Sintáctico: estructura de instrucciones inválida según la gramática.
- Semántico: errores de tipos, variables no declaradas u otras reglas del lenguaje.
- Advertencia: situaciones que no detienen el análisis pero conviene revisar.



Consola | Errores (12)

TIPO ERROR	FILA	COLUMNA	DESCRIPCIÓN
Semántico	6	1	Variable ya declarada: 'resultado'
Semántico	13	19	Argumento 1 incompatible: se esperaba entero, se recibió cadena
Semántico	16	13	Función no declarada: 'funcionInexistente'
Semántico	19	13	Identificador no declarado: 'variable_fantasma'
Semántico	22	1	No se puede asignar cadena a entero

8.4 Marcado de errores en el editor

Las líneas donde se detectó un error se resaltan directamente en el editor, de modo que sea más fácil ubicarlas sin necesidad de revisar la tabla completa.

```
VARIABILES>
esto resultado : numerus 0;
esto resultado : textum "duplicado";       // ERROR 1: redeclaración con tipo distinto
esto activo : falsus;

MAIOR>
>> "Iniciando prueba con errores...";

// ERROR 2: argumento String a función que espera entero
resultado = sumar("hola", 5);

// ERROR 3: función no existe
resultado = funcionInexistente(5);

// ERROR 4: variable no declarada
resultado = variable_fantasma + 5;

// ERROR 5: asignar String a variable numerus
resultado = "esto es texto";

// ERROR 6: clase inexistente
esto robot : novus Robot("R2D2", 100);

// ERROR 7: campo inexistente en Estudiante
esto alumno : novus Estudiante("Ana", 2021001);
esto edad_alumno : numerus 0;
edad_alumno = alumno.edad;                 // "Estudiante" no tiene campo "edad"
```

8.5 Tabla de símbolos

La tabla de símbolos muestra todos los identificadores registrados durante el análisis semántico. Cada entrada contiene: el nombre del símbolo, su categoría (variable, función, clase, atributo, parámetro, estructura, campo, constructor o método), el tipo de dato, el ámbito donde fue declarado, la línea y columna de su declaración, y si fue inicializado. Para atributos de clases y campos de estructuras se

incluye el offset en palabras (posición dentro del objeto en el heap). Esta información es útil para verificar que el compilador resolvió correctamente los tipos y las posiciones de memoria.

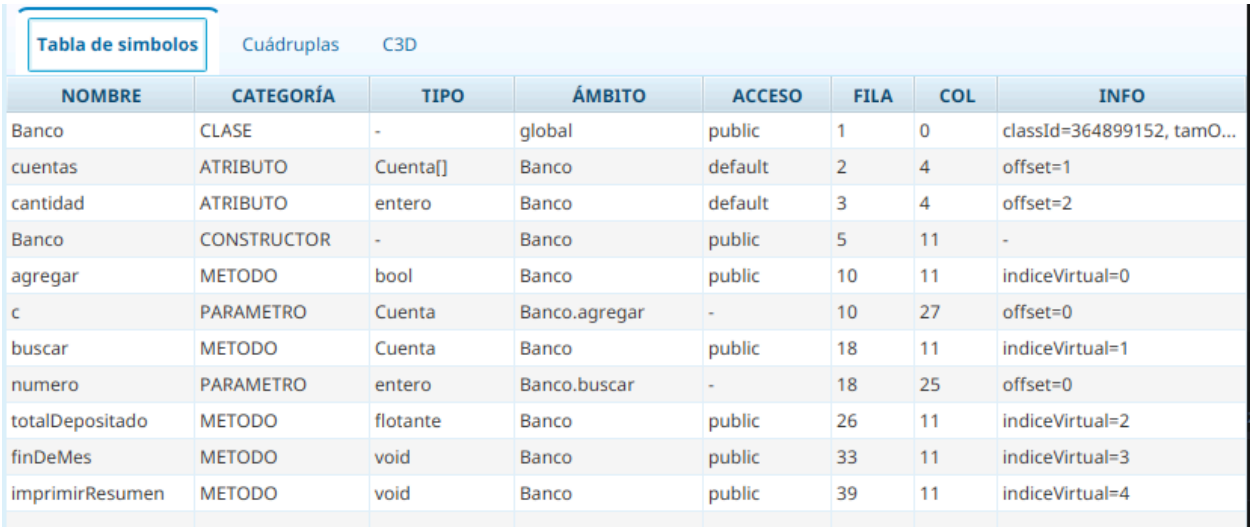


Tabla de simbolos | Cuádruplas | C3D

NOMBRE	CATEGORÍA	TIPO	ÁMBITO	ACCESO	FILA	COL	INFO
Banco	CLASE	-	global	public	1	0	classId=364899152, tamO...
cuentas	ATRIBUTO	Cuenta[]	Banco	default	2	4	offset=1
cantidad	ATRIBUTO	entero	Banco	default	3	4	offset=2
Banco	CONSTRUCTOR	-	Banco	public	5	11	-
agregar	METODO	bool	Banco	public	10	11	indiceVirtual=0
c	PARAMETRO	Cuenta	Banco.agregar	-	10	27	offset=0
buscar	METODO	Cuenta	Banco	public	18	11	indiceVirtual=1
numero	PARAMETRO	entero	Banco.buscar	-	18	25	offset=0
totalDepositado	METODO	flotante	Banco	public	26	11	indiceVirtual=2
finDeMes	METODO	void	Banco	public	33	11	indiceVirtual=3
imprimirResumen	METODO	void	Banco	public	39	11	indiceVirtual=4

9. Visualización del Código Intermedio (C3D)

9.1 ¿Qué es el código de tres direcciones?

El código de tres direcciones (C3D) es una representación intermedia que el compilador genera a partir del árbol sintáctico abstracto (AST) después del análisis semántico. Cada instrucción tiene como máximo un operador y hasta tres operandos (dos fuentes y un destino). Esta representación simplifica la traducción a código máquina porque cada instrucción se corresponde casi directamente con una instrucción del procesador.

Ejemplo de C3D para la expresión "x = a + b * c":

t0 = b * c t1 = a + t0 x = t1

9.2 Tabla de cuádruplas

La tabla de cuádruplas se presenta en el panel inferior del IDE después de generar el C3D. Cada fila contiene:

Columna	Descripción
#	Número secuencial de la cuádrupla
Operación	El tipo de instrucción (asignación, binaria, goto, if_false, call, param, return, etc.)
Operando 1	Primer argumento (variable, temporal o literal)
Operando 2	Segundo argumento (si aplica)
Resultado	Destino donde se almacena el resultado (temporal o variable)

10 ¿Qué es RISC-V?

RISC-V es una arquitectura de conjunto de instrucciones (ISA) abierta y extensible. El compilador genera código para la extensión base RV32I (enteros de 32 bits) más la extensión F (flotantes de precisión simple). El código generado se ejecuta en el emulador RARS, que simula un procesador RISC-V completo con memoria, registros y servicios de sistema (ecall).

10.1 Cómo generar el archivo .s

1. Asegúrese de que el archivo activo sea un .pig (punto de entrada del programa) o un .z/.y individual.
2. Haga clic en el botón "RISC-V" de la barra de herramientas, o vaya a Ejecutar > Generar RISC-V.
3. El sistema ejecuta automáticamente el análisis completo (léxico, sintáctico, semántico), genera el C3D y lo traduce a RISC-V.
4. Si no hay errores, el archivo .s se genera en la carpeta de salida del proyecto y se muestra en el panel de código RISC-V.
5. Si hay errores semánticos, se muestran en la tabla de errores y no se genera el .s.

10.2 Estructura del archivo .s generado

El archivo .s tiene la siguiente estructura:

Sección	Contenido
.data	Literales de cadena (.string), literales flotantes (.word con bits IEEE 754), y la zona del heap (heap_ptr + heap_mem)
.text	Directiva .globl main, seguida del código de cada función: prólogo (reservar frame, guardar ra y fp), cuerpo (instrucciones traducidas de las cuádruplas) y epílogo (restaurar ra y fp, liberar frame, retornar)

10.3 Cómo ejecutar el .s en RARS

Para ejecutar el código generado en el emulador RARS:

1. Descargue RARS desde su repositorio oficial (rars1_6.jar o superior).
2. Abra una terminal en la carpeta donde se encuentra el archivo .s generado.
3. Ejecute el siguiente comando:

java -jar rars1_6.jar MiPrograma.s

4. El programa se ejecuta y muestra la salida en la terminal. Si el programa espera entrada del usuario (readln), escriba el valor y presione Enter.
5. Si ocurre un error en tiempo de ejecución (por ejemplo, dirección no alineada o fuera de rango), RARS muestra la línea del .s donde ocurrió y una descripción del error.




```
└─> java -jar rars1_6.jar Batalla.s
RARS 1.6  Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

=== Preparacion ===
Unidad nula
[G1][A1][..][..][T2][G2][..]
=== Combate ===
Combate invalido
Aquiles fuera de alcance
Robin fuera de alcance
Robin golpea a Orco con 4 -> vida 6
Robin golpea a Orco con 4 -> vida 3
Robin golpea a Orco con 0 -> vida 3
Movimiento invalido
Aquiles golpea a Goliat con 4 -> vida 7
Goliat golpea a Aquiles con 6 -> vida 7
Fuego amigo bloqueado
Aquiles golpea a Goliat con 5 -> vida 3
Goliat vida=5 blindaje=1 pos=4
  energia=1
Goliat golpea a Aquiles con 7 -> vida 1
Aquiles golpea a Goliat con 6 -> vida 0
Goliat eliminado
Aquiles golpea a Orco con 7 -> vida 0
Orco eliminado
=== Resultado ===
Aquiles ruge con furia 4
[..][..][A1][..][G1][..][..]
Aquiles vida=1 blindaje=1 pos=4
Unidades en tablero: 2
Bono cercania: 1
Arbitro ve Refuerzo en 6
Sanciones: 1
```

10.4 Manejo de memoria en el código generado

El código RISC-V generado utiliza dos zonas de memoria:

- Pila (stack): cada llamada a función reserva un frame en la pila. El frame contiene la dirección de retorno (ra), el frame pointer del llamador (fp), y un slot de 4 bytes para cada parámetro,

variable local y temporal. Todos los datos se acceden como offsets negativos desde fp. La pila crece hacia direcciones menores.

- Heap: los objetos (instancias de clases de Z), las estructuras (de Y?), los arreglos y las cadenas concatenadas en tiempo de ejecución se crean en el heap. Se utiliza un bump allocator: un puntero global (heap_ptr) que avanza con cada asignación y nunca retrocede. No hay liberación de memoria.

10.5 Servicios del sistema (ecall)

El código generado utiliza los siguientes servicios de RARS para entrada/salida:

a7	Servicio	a0 / fa0
1	Imprimir entero	a0 = valor
2	Imprimir flotante	fa0 = valor
4	Imprimir cadena	a0 = dirección
5	Leer entero	resultado en a0
6	Leer flotante	resultado en fa0
8	Leer cadena	a0 = buffer, a1 = tamaño
10	Terminar programa	(sin argumentos)
11	Imprimir carácter	a0 = código ASCII
12	Leer carácter	resultado en a0